

ENTERPRISE ARCHITECTURE

BEST PRACTICES & JARGON GUIDE

Pitch to Retire — Terminology, Standards & Golden Rules

The definitive reference guide for EA practitioners, business stakeholders, and delivery teams. Part 1 covers the 20 EA Golden Rules. Part 2 is an A–Z jargon buster of 60+ enterprise architecture terms, acronyms, and frameworks used across the full capability lifecycle.

PART 1 20 EA Golden Rules Best Practices

PART 2 60+ Terms Jargon Buster

May 2026

EA Practice Reference

v1.0

Part 1 — The 20 EA Golden Rules

These twenty principles represent the distilled best practices of enterprise architecture across strategy, governance, delivery, and portfolio management. They are applicable across all seven lifecycle stages.

1 Architecture Is an Enabling Function, Not a Gatekeeper

The EA exists to accelerate value delivery, not to slow it down. Every governance process must add more value than the friction it creates. If your ARB is seen as a bureaucratic blocker, the architecture function has failed.

- ✓ Make governance fast, transparent, and outcome-focused
- ✓ Publish architecture principles and standards so teams can self-serve
- ✓ Measure EA value by business outcomes, not documents produced
- ✗ Don't make the ARB a committee that exists to say no
- ✗ Don't add process steps unless they prevent a real, recurring failure

2 Never Approve a Solution Before Understanding the Problem

The most expensive architectural mistakes begin when a sponsor arrives with a pre-chosen solution and EA rubber-stamps it. The pitch stage exists to validate the problem, not to confirm the sponsor's preferred answer.

- ✓ Always start with a problem statement signed off by the sponsor
- ✓ Challenge solution assumptions respectfully and with data
- ✓ Run the build/buy/reuse assessment before any technology is selected
- ✗ Don't let relationship pressure bypass the intake assessment
- ✗ Don't let urgency substitute for understanding

3 Technical Debt Is a Business Risk — Treat It as One

Technical debt is not a technical problem. It is a business liability that compounds over time, reducing agility, increasing risk, and raising operating costs. The EA must make debt visible to business leaders in business language.

- ✓ Maintain a technical debt register with business impact scores
- ✓ Present debt as risk-adjusted cost in every portfolio review
- ✓ Set a debt reduction target as a formal KPI (e.g. 10% per quarter)
- ✗ Don't allow 'we'll fix it later' without a named owner and date
- ✗ Don't bury debt in technical jargon — quantify it in dollars and risk

4 Every Architecture Decision Must Have a Record

Architecture Decision Records (ADRs) are the institutional memory of the organisation's technology choices. Without them, the same debates happen repeatedly and institutional knowledge walks out the door with departing staff.

- ✓ Write ADRs in plain language: context, options, decision, consequences
- ✓ Publish ADRs in a shared, searchable repository accessible to all engineers
- ✓ Review ADRs at each annual rationalisation cycle — some will need updating
- ✗ Don't allow verbal ARB decisions — all decisions must be written
- ✗ Don't write ADRs that describe what was built — write why

5 Standards Must Be Enforced, Not Just Published

A technology standard that is not enforced is a wish list. Governance without teeth creates the illusion of control while architectural sprawl continues unchecked. Enforcement must be automated wherever possible.

- ✓ Implement architecture fitness functions in delivery pipelines
- ✓ Include standards compliance in the pre-production sign-off gate
- ✓ Track and report the architecture adoption rate as a KPI
- ✗ Don't publish standards and hope teams read them
- ✗ Don't grant exceptions without a named owner and resolution date

6 Design for the Organisation You Are Becoming, Not the One You Are

The architecture must support the organisation's strategic trajectory, not just its current state. A three-to-five year architecture roadmap grounded in strategic themes prevents constant reactive rebuilding.

- ✓ Maintain a 1/3/5 year architecture roadmap aligned to corporate strategy
- ✓ Review and recalibrate roadmaps quarterly, not just annually
- ✓ Map all major initiatives to at least one strategic theme
- ✗ Don't design for today's volume if the strategy calls for 10x growth
- ✗ Don't let the roadmap become a wish list with no prioritisation

7 Build/Buy/Reuse — In That Order of Scrutiny

Before committing to a build or a buy, the EA must genuinely explore reuse. The most sustainable architectures are built on a lean portfolio of well-used platforms, not a collection of specialist point solutions.

- ✓ Check the existing portfolio before every intake assessment
- ✓ Challenge vendor selections with a reuse analysis
- ✓ Track and reduce the number of duplicate capabilities in the portfolio
- ✗ Don't allow different parts of the business to solve the same problem independently
- ✗ Don't buy a new tool when an existing one, properly configured, would suffice

8 Non-Functional Requirements Are Not Optional

NFRs define the quality of the system, not just its features. An application that works but fails under load, cannot be recovered in an incident, or leaks data is a liability, not an asset. NFRs must be defined before design, not discovered after launch.

- ✓ Define availability, RTO, RPO, performance, and scalability at design time
- ✓ Make NFR testing mandatory for every pre-production gate
- ✓ Tier NFRs by system criticality (Tier 1/2/3 requirements differ)
- ✗ Don't treat NFRs as 'nice to haves' to be addressed post-launch
- ✗ Don't allow delivery teams to set their own NFRs without EA validation

9 The 30% Review Is Your Most Cost-Effective Governance Investment

A structural architectural mistake caught at 30% of delivery costs a fraction of what it costs to fix at 90% or in production. The 30% architecture review is not a status update — it is a diagnostic intervention.

- ✓ Schedule the 30% review as a mandatory delivery milestone

✓ Focus on structural decisions: system boundaries, integration patterns, data ownership

✓ Document findings as formal exceptions requiring resolution before 70% gate

✗ Don't cancel the 30% review due to delivery pressure

✗ Don't allow it to become a PowerPoint presentation — review actual implementation

10 Observability Is Architecture, Not an Afterthought

A system you cannot see is a system you cannot manage. Logging, monitoring, alerting, and distributed tracing must be designed in from the start and validated before any system enters production.

✓ Include observability requirements in the SAD as a mandatory NFR

✓ Validate the observability stack in the pre-production compliance sign-off

✓ Define SLO dashboards for every Tier-1 system

✗ Don't allow a system to go live without a working monitoring dashboard

✗ Don't leave observability to the operations team to figure out post-launch

11 Application Rationalisation Is a Business Conversation, Not a Technical One

Business owners must be active participants in rationalisation decisions. The EA provides the technical health score; the business provides the value score. Neither can make a rationalisation decision alone.

- ✓ Run joint scoring workshops with business owners annually
- ✓ Use usage data, integration counts, and revenue attribution to anchor value discussions
- ✓ Present rationalisation recommendations in business language, not technical
- ✗ Don't make rationalisation decisions without business buy-in
- ✗ Don't let technical health scores dominate — a healthy system with no business value is still waste

12 Retirement Is a Project — Resource and Govern It as One

Decommissioning a system is as complex as deploying a new one. Dependency resolution, data migration, contract termination, and access removal all require dedicated effort. Unmanaged retirements create orphaned integrations, data breaches, and zombie cost.

- ✓ Give every retirement a project code, a project manager, and a timeline
- ✓ Mandate 90-day notice periods to all dependent teams
- ✓ Make access removal the final mandatory step in every decommission runbook
- ✗ Don't assume switching off a system is simply a matter of turning it off
- ✗ Don't allow a retirement to start without a complete dependency map

13 Use Your Technology Radar to Prevent Sprawl

The Technology Radar (Adopt / Trial / Hold / Retire) is the EA's most powerful tool for controlling technology sprawl. Every technology in the estate should have a published position, reviewed annually.

- ✓ Publish and socialise the Technology Radar organisation-wide
- ✓ Reject any ARB submission proposing a technology on the 'Hold' or 'Retire' list
- ✓ Review and publish an updated radar at least annually
- ✗ Don't let teams adopt new technologies without ARB awareness
- ✗ Don't allow the radar to become stale — an outdated radar is worse than none

14 Strategic Alignment Is the EA's North Star

Every architecture decision, every investment recommendation, and every rationalisation choice must be traceable to at least one strategic theme. An EA who cannot articulate the strategic rationale for a decision is operating as a technical function, not a strategic one.

- ✓ Map every initiative to a strategic theme at intake

✓ Report strategic alignment % as a headline EA KPI

✓ Challenge any initiative that cannot be mapped to a strategic objective

✗ Don't approve initiatives that have no strategic justification

✗ Don't let operational pressure drive architecture decisions that contradict strategy

15 EA Metrics Must Tell a Business Story

Activity metrics (reviews completed, documents produced) do not demonstrate value. Business outcome metrics (TCO reduced, time to market improved, risk incidents prevented) do. The EA must measure and communicate in the language of business.

✓ Maintain a dual dashboard: EA activity metrics + business outcome metrics

✓ Present EA value in terms of cost saved, risk reduced, and speed improved

✓ Tie EA KPIs to the corporate OKR cycle

✗ Don't report architecture review counts as a measure of impact

✗ Don't allow the EA function to become invisible by failing to communicate value

16 Stakeholder Trust Is Built Through Consistency, Not Perfection

Business leaders do not trust EA because it produces perfect architectures — they trust it because it is consistent, transparent, and delivers what it promises. Reliability over time is the foundation of strategic influence.

- ✓ Meet every governance commitment and communicate early when you cannot
- ✓ Publish architecture principles and stick to them consistently
- ✓ Follow up on every ARB condition with the owner — do not let them drift
- ✗ Don't make governance exceptions under business pressure without documenting the risk
- ✗ Don't change architectural positions without communicating the reasoning

17 Cloud Is a Deployment Model, Not a Strategy

Cloud adoption must be governed by the same architectural rigour as any other technology decision. 'Move to cloud' is not a strategy. The EA must define a Cloud Adoption Framework that covers placement policy, cost governance, security posture, and operating model.

- ✓ Define and publish a cloud placement policy (public/private/hybrid criteria)
- ✓ Implement FinOps governance to prevent cloud cost sprawl
- ✓ Validate cloud architecture against the Well-Architected Framework for every design
- ✗ Don't allow teams to provision cloud resources outside of approved patterns
- ✗ Don't mistake cloud migration for modernisation — a legacy app on cloud is still legacy

18 Security Architecture Is Non-Negotiable

Security is not a feature to be added later. The EA must ensure that every design incorporates security-by-design principles: zero trust posture, least privilege access, encryption at rest and in transit, and comprehensive audit logging.

- ✓ Make security architecture review mandatory at the Design gate
- ✓ Require penetration testing or DAST scanning before every production gate
- ✓ Track open security architecture findings by severity as a portfolio KPI
- ✗ Don't allow 'we'll add security hardening post-launch'
- ✗ Don't treat security as the CISO's problem — it is a shared EA responsibility

19 Integration Patterns Are the Connective Tissue of the Enterprise

Poorly governed integration is the root cause of most enterprise architectural crises. Point-to-point integrations create fragile, unmaintainable webs. The EA must enforce approved integration patterns (API-first, event-driven, canonical data model) as non-negotiable standards.

- ✓ Publish and enforce an enterprise integration pattern catalogue

- ✓ Mandate API-first design for all new integrations
- ✓ Maintain an integration dependency register and review it quarterly
- ✗ Don't allow point-to-point integrations between production systems
- ✗ Don't allow integrations to be built without API contracts reviewed by EA

20 Great EAs Enable Change — They Do Not Resist It

The most common failure mode of enterprise architecture functions is becoming the department of 'no'. EA must continuously evolve its standards, patterns, and frameworks to embrace new paradigms — AI, platform engineering, composable architecture — before they arrive uninvited through the back door.

- ✓ Maintain a formal EA innovation backlog reviewed quarterly
- ✓ Pilot new paradigms through controlled trials on low-risk initiatives
- ✓ Invite emerging technology perspectives into the Technology Radar process
- ✗ Don't dismiss new approaches without structured evaluation
- ✗ Don't allow the EA function to become a museum of past decisions

Part 2 — EA Jargon Buster (A–Z)

A reference glossary of 60+ terms, acronyms, and frameworks used in Enterprise Architecture practice. Each entry includes a plain-language definition and a 'in practice' note showing how the term is used in real EA conversations.

CATEGORY: GOVERNANCE & PROCESS

ADR**Architecture Decision Record**

A short document capturing the context, options considered, decision made, and consequences of a significant architectural choice. ADRs create an auditable trail of architectural reasoning.

In practice: "The ADR for the API gateway selection explains why we chose Kong over AWS API Gateway — pull it up before we revisit that decision."

ARB**Architecture Review Board**

A governance body that reviews and approves architecture proposals, sets standards, and manages architectural risk across the enterprise. The ARB is the EA's primary decision-making forum.

In practice: "This proposal needs to go to the ARB before we commit budget — it touches three Tier-1 systems."

AIA**Architecture Intake Assessment**

The output document of the Pitch stage. A structured 1–2 page assessment covering the business problem, landscape findings, build/buy/reuse recommendation, and initial risk flags.

In practice: "The AIA shows two existing systems could satisfy this need — let's validate that with the business owner before writing a business case."

AFC**Architecture Fitness Function**

An automated test or metric that objectively assesses whether an architecture characteristic (e.g. coupling, security posture) conforms to defined standards. Borrowed from evolutionary architecture practice.

In practice: "We have an AFC in the CI pipeline that fails the build if any service exceeds the approved response time threshold."

SAD**Solution Architecture Document**

The primary design artefact produced during the Design stage. Covers functional requirements, non-functional requirements, integration architecture, data flows, and security design.

In practice: "The SAD for this integration needs a sequence diagram — the ARB won't approve without seeing the message flow."

CoE**Centre of Excellence**

A team of subject matter experts who define, maintain, and promote best practices in a specific domain (e.g. Data CoE, Cloud CoE, Integration CoE). CoEs feed into EA standards.

In practice: "The Cloud CoE owns the Well-Architected review process — loop them in before the Design gate."

CATEGORY: ARCHITECTURE FRAMEWORKS

TOGAF	The Open Group Architecture Framework The most widely adopted enterprise architecture framework. Provides the Architecture Development Method (ADM) — a cycle of phases from architecture vision through implementation governance. <i>In practice: "Our EA lifecycle is TOGAF-inspired, but we've simplified the ADM to fit our delivery cadence."</i>
ArchiMate	Architecture Modelling Language An open standard visual notation for enterprise architecture diagrams. Covers business, application, and technology layers with standardised symbols and relationships. <i>In practice: "Use ArchiMate notation for the context diagram — it needs to be consistent with our EA repository."</i>
C4 Model	Context, Container, Component, Code Model A lightweight, developer-friendly notation for software architecture diagrams with four levels of abstraction: System Context, Containers, Components, and Code. <i>In practice: "A C4 Level 1 context diagram is mandatory in every SAD — it shows external actors and system boundaries."</i>
ITIL	IT Infrastructure Library A framework of best practices for IT service management (ITSM). Relevant to EA in the Operate stage — particularly incident management, change management, and service design. <i>In practice: "The decommission plan needs to follow the ITIL change management process — raise an RFC before you switch anything off."</i>
Zachman	Zachman Framework A 6x6 matrix framework for organising architectural artefacts by stakeholder perspective (What, How, Where, Who, When, Why) and abstraction level. Useful for classifying EA documentation. <i>In practice: "The Zachman framework helps us ensure we've addressed every dimension of the architecture, not just the technical view."</i>
SAFe	Scaled Agile Framework A framework for applying agile and lean principles at enterprise scale. The EA function must align architecture governance with SAFe's PI Planning and ART cadences in SAFe organisations. <i>In practice: "Our EA governance checkpoints are now embedded in the SAFe PI Planning events rather than separate review meetings."</i>

CATEGORY: DESIGN & MODELLING

API	Application Programming Interface A defined contract specifying how software components communicate. In EA, 'API-first design' means integration contracts are defined before implementation begins. <i>In practice: "The API contract for this integration must be reviewed by EA before the team starts building — the schema needs to follow our canonical data model."</i>
-----	--

EDA	Event-Driven Architecture An architectural pattern where services communicate through asynchronous events (messages) rather than synchronous API calls. Improves decoupling and scalability. <i>In practice: "We're moving the order processing pipeline to EDA — it'll decouple the inventory service from the fulfilment service."</i>
SoR / SoE	System of Record / System of Engagement SoR: the authoritative source of truth for a data entity (e.g. CRM for customer data). SoE: systems through which users interact with that data. Clarifying ownership prevents data duplication. <i>In practice: "The CRM is the SoR for customer contact data — any system that holds a copy must sync from it, not create its own record."</i>
CDM	Canonical Data Model An enterprise-wide data model that defines standard data structures and terminology shared across systems. Prevents the proliferation of inconsistent data representations. <i>In practice: "The new integration must use the CDM for the product entity — we can't have eight different definitions of 'product' in the estate."</i>
NFR	Non-Functional Requirement A requirement specifying quality attributes rather than functional behaviour — availability, performance, scalability, security, maintainability, and recoverability. <i>In practice: "The NFRs for this Tier-1 service require 99.9% availability and an RTO of 4 hours — the infrastructure design must reflect that."</i>
RTO / RPO	Recovery Time Objective / Recovery Point Objective RTO: the maximum acceptable time to restore a system after a failure. RPO: the maximum acceptable data loss measured in time. Both must be defined for every production system. <i>In practice: "The business confirmed an RTO of 2 hours and RPO of 15 minutes for the payments platform — that requires active-active deployment."</i>

CATEGORY: TECHNOLOGY STRATEGY

TCO	Total Cost of Ownership The full lifecycle cost of a technology asset including acquisition, implementation, operation, support, and decommission. TCO analysis prevents underestimating the true cost of a technology choice. <i>In practice: "The SaaS option looks cheaper upfront, but when you include integration costs and data migration, the 5-year TCO is higher than the on-prem build."</i>
Tech Radar	Technology Radar A framework (popularised by ThoughtWorks) that classifies technologies into four rings: Adopt (recommended), Trial (experiment cautiously), Hold (pause new adoption), Retire (decommission existing use). <i>In practice: "Check the tech radar before proposing any new technology in an ARB submission — if it's on 'Hold', you'll need a compelling case."</i>

FinOps	Financial Operations for Cloud A practice that brings financial accountability to cloud spending by creating shared ownership between engineering, finance, and business teams to optimise cloud cost. <i>In practice: "The cloud bill tripled last quarter with no visibility into which team is responsible — we need to implement FinOps governance."</i>
IaC	Infrastructure as Code The practice of managing and provisioning infrastructure through machine-readable configuration files rather than manual processes. Enables repeatable, auditable deployments. <i>In practice: "All cloud infrastructure must be provisioned via IaC — no manual console-clicks in production environments."</i>
Well-Architected	Well-Architected Framework A framework (originally from AWS, now available for Azure and GCP) that evaluates cloud architectures across five pillars: operational excellence, security, reliability, performance, and cost optimisation. <i>In practice: "Run a Well-Architected review before the Design gate sign-off for any cloud-native solution."</i>
Zero Trust	Zero Trust Security Model A security architecture principle that assumes no user or system is trustworthy by default, requiring continuous verification of identity and least-privilege access regardless of network location. <i>In practice: "The new microservices platform must implement zero trust — no implicit trust between services, even inside the VPC."</i>

CATEGORY: RISK & COMPLIANCE

EOL	End of Life The point at which a vendor stops providing security patches and support for a software version or product. Running EOL technology is an immediate security and compliance risk. <i>In practice: "That middleware version hits EOL in 8 months — we need to start the upgrade project now or we'll be in breach of our security policy."</i>
DR	Disaster Recovery The set of policies, tools, and procedures to recover or continue vital technology infrastructure and systems following a disaster. Defined by RTO and RPO targets. <i>In practice: "The DR drill for the payments platform is scheduled for next month — EA needs to sign off the test plan first."</i>
DAST	Dynamic Application Security Testing A security testing method that analyses a running application from the outside to find vulnerabilities (SQL injection, XSS, etc.). Required as part of the pre-production security gate. <i>In practice: "DAST scan results came back with three medium findings — all must be resolved before the production compliance sign-off."</i>

GDPR	General Data Protection Regulation EU regulation governing the processing of personal data. EA must ensure architecture decisions address data residency, right to erasure, consent management, and privacy by design. <i>In practice: "The data migration plan for the decommission needs Legal sign-off — some of that data may be subject to GDPR retention rules."</i>
CIA Triad	Confidentiality, Integrity, Availability The three core principles of information security. Every security architecture design must address all three dimensions. <i>In practice: "Walk me through how this design addresses the CIA triad — I don't see any integrity controls on the data in transit."</i>
RBAC	Role-Based Access Control An approach to access management where permissions are assigned to roles rather than individuals. RBAC must be defined in the SAD and validated before production go-live. <i>In practice: "The SAD needs to define the RBAC model — who has read, write, and admin access and under what conditions."</i>

CATEGORY: MEASUREMENT & VALUE

OKR	Objectives and Key Results A strategic goal-setting framework pairing an inspirational objective with 2–4 measurable key results. OKRs run on quarterly cycles and drive organisational alignment. <i>In practice: "The EA team's Q3 OKR is to reduce the EOL technology percentage from 12% to below 5% — here are the three key results we'll track."</i>
KPI	Key Performance Indicator A specific, quantifiable metric used to evaluate performance against a defined target. KPIs measure the ongoing health of processes and initiatives. <i>In practice: "Our headline EA KPI for this quarter is architecture adoption rate — currently at 62%, target is 85% by year end."</i>
NPS	Net Promoter Score A metric measuring stakeholder loyalty and satisfaction by asking how likely they are to recommend EA services. Tracked as a Stakeholder Value KPI. <i>In practice: "Our EA NPS from business leaders was 38 last year — improving it to 55 is an OKR for the next two quarters."</i>
ROI	Return on Investment $[(\text{Financial Gain} - \text{Cost of Investment}) / \text{Cost of Investment}] \times 100$. Used to justify architecture decisions and investment recommendations to business stakeholders. <i>In practice: "The rationalisation programme delivered a 22% ROI in year one — that's the number we lead with in the board update."</i>

MTTR / MTTF	Mean Time to Recover / Mean Time to Failure MTTR: average time to restore a failed system. MTTF: average time between system failures. Both are operational reliability KPIs tracked in the Operate stage. <i>In practice: "The payments platform MTTR is 47 minutes — we need to get that below 30 minutes to meet the new SLA commitment."</i>
Maturity Model	Architecture Capability Maturity Model A framework that assesses the capability maturity of an architecture function or business domain across levels from Initial (ad hoc) to Optimising (continuously improving). <i>In practice: "The rationalisation assessment put the customer data domain at maturity Level 2 — our roadmap targets Level 3 by next year."</i>

CATEGORY: DELIVERY & AGILITY

CI/CD	Continuous Integration / Continuous Delivery A set of practices where code changes are automatically built, tested, and deployed. Architecture fitness functions should be embedded in CI/CD pipelines to enforce standards. <i>In practice: "The architecture compliance checks run automatically in the CI pipeline — a failed fitness function blocks the merge."</i>
MVP	Minimum Viable Product The smallest increment of a product that delivers value and can be tested with users. EAs must ensure MVPs are architecturally sound, not just functionally minimal. <i>In practice: "The MVP is scoped correctly from a feature perspective, but the team has skipped the security architecture — that's not negotiable even for a pilot."</i>
PBI	Product Backlog Item A unit of work in an agile delivery framework (Scrum/SAFe). Architecture requirements (NFRs, guardrails) should be represented as PBIs to be prioritised and delivered. <i>In practice: "The observability requirements from the SAD should be logged as PBIs in the backlog — they need to be delivered before the pre-production gate."</i>
Spike	Architecture Spike A time-boxed investigation (typically 1–2 sprints) used to resolve architectural uncertainty before committing to a design. Spikes reduce risk in novel or complex domains. <i>In practice: "We don't have enough confidence in the event streaming performance — let's run a 2-sprint spike before we commit to that architecture."</i>
Platform Engineering	Platform Engineering The discipline of building and maintaining internal developer platforms (IDPs) that enable delivery teams to self-serve infrastructure, tooling, and shared services. EA defines the platform strategy. <i>In practice: "Our platform engineering team is building the internal developer portal — EA needs to define the approved service catalogue it will expose."</i>

Paved Road**Paved Road / Golden Path**

A pre-approved, well-supported set of technology choices and deployment patterns that teams are encouraged (but not always required) to follow. Reduces cognitive overhead and enforces standards.

In practice: "The paved road for new microservices is: Node.js or Java, Kubernetes deployment, Kong API gateway, Datadog observability — teams can deviate with an ARB exception."

CATEGORY: PORTFOLIO MANAGEMENT**App Rat****Application Rationalisation**

The process of evaluating every system in the portfolio on business value and technical health to determine its lifecycle disposition: Retain, Invest, Migrate, Consolidate, or Retire.

In practice: "The annual app rat exercise identified 14 candidates for retirement — that's £2.4M in run cost we can eliminate."

Disposition**Lifecycle Disposition**

The decision made at the rationalisation stage about a system's future: Retain, Invest, Migrate, Consolidate, or Retire. Each disposition has a defined set of subsequent actions.

In practice: "The disposition for the legacy CRM is 'migrate' — we'll transition its capabilities to Salesforce over the next 18 months."

Capability Map**Business Capability Map**

A structured visual representation of what a business does (its capabilities) independent of how it does it. Used in EA to map technology investments to business outcomes.

In practice: "Let's map this initiative to the capability map — it looks like it touches both the Order Management and Customer Fulfilment capabilities."

Integration Register**Integration Dependency Register**

A catalogue of all integration points between systems — who consumes what, via which protocol, with what SLA. Essential for impact analysis and retirement planning.

In practice: "Before we decommission the legacy product catalogue, check the integration register — last count there were 23 upstream consumers."

Zombie System**Zombie System**

A production system that is no longer actively used but continues to draw operating cost because no one has formally retired it. A symptom of weak portfolio governance.

In practice: "That reporting system hasn't had a login in 18 months but we're still paying £40k a year in hosting — it's a zombie, let's start the retirement process."

Shadow IT**Shadow IT**

Technology adopted by business teams without EA or IT knowledge or approval. Shadow IT bypasses governance, creates security risks, and generates invisible technical debt.

In practice: "The marketing team has been using a SaaS analytics tool that processes customer PII — we need to get it through an urgent ARB review or shut it down."